PLOS ONE

RESEARCH ARTICLE

An integrated blockchain and IPFS-based solution for secure and efficient source code repository hosting using middleman approach

Md. Rafid Haque[1], Sakibul Islam Munna[1], Sabbir Ahmed[1]*, Md. Tahmid Islam[1], Md Mehedi Hassan Onik[1,2], A.B.M. Ashikur Rahman[1,3]

1 Department of Computer Science and Engineering, Islamic University of Technology (IUT), Boardbazar, Gazipur, Bangladesh, **2** School of IT, Deakin University, Waurn Ponds, Victoria, Australia, **3** Department of ICS, King Fahd University of Petroleum & Minerals, Dhahran, Saudi Arabia

* sabbirahmed@iut-dhaka.edu

OPEN ACCESS

Citation: Haque R, Munna SI, Ahmed S, Islam T, Hassan MM, Rahman A (2025) An integrated blockchain and IPFS-based solution for secure and efficient source code repository hosting using middleman approach. PLoS One 20(9): e0331131. https://doi.org/10.1371/journal.pone.0331131

Editor: Yang (Jack) Lu, Beijing Technology and Business University, CHINA

Received: September 27, 2024

Accepted: August 11, 2025

Published: September 3, 2025

Peer Review History: PLOS recognizes the benefits of transparency in the peer review process; therefore, we enable the publication of all of the content of peer review and author responses alongside final, published articles. The editorial history of this article is available here: https://doi.org/10.1371/journal.pone.0331131

Data availability statement: All relevant data for this study are within the paper, its Supporting Information files, and publicly

Abstract

Centralized version control systems (VCS) are vital for software development but pose risks of data loss and ownership disputes. While blockchain offers a decentralized alternative, existing solutions are often hindered by high latency, compromising the real-time collaboration essential for modern workflows. This study introduces a novel hybrid architecture combining the security of the Ethereum blockchain and the InterPlanetary File System (IPFS) with two key contributions: 1) Shamir's Secret Sharing (SSS) to create a trust-minimized model for key distribution, and 2) an authoritative-first, optimistic-fallback retrieval protocol utilizing a temporary middleware to decouple the user experience from blockchain confirmation delays. We implemented a full prototype and conducted a comprehensive performance evaluation on the public Sepolia testnet. Our results demonstrate that this architecture not only provides a secure, auditable, and resilient platform for source code hosting but also achieves highly competitive user-perceived performance. Our user-perceived push time reduces submission latency by up to 49% compared to a standard git push for common repository sizes, proving that a well-designed decentralized VCS can balance the core tenets of security and decentralization with the practical need for speed and efficiency.

Introduction

Software development projects often rely on version control systems (VCS) to track and manage their code and files. However, most existing VCS are centralized, which means that they depend on a single authority or service provider that can pose risks of data loss, security breaches, and ownership disputes [1–3]. Therefore, there is a need for a decentralized, reliable, and secure solution for code repository hosting and governance. Blockchain technology is a promising candidate for such a solution, as it enables a distributed ledger that is immutable, transparent, and consensus-based, without any trusted intermediaries [4,5].

available from the Zenodo repository (https://doi.org/10.5281/zenodo.15700465).

Funding: The author(s) received no specific funding for this work.

Competing interests: The authors have declared that no competing interests exist.

The use of self-executing smart contracts can further automate processes, reduce the need for intermediaries, and enforce predefined rules for digital transactions with high security [6–12].

While blockchain has been successfully applied to enhance trust and transparency in domains like supply chain management [13–17] and healthcare [18–23], its application to performance-sensitive domains like version control faces a critical obstacle. The primary challenge is the inherent latency of on-chain transactions. A system requiring every action to await a blockchain confirmation, which can take several seconds or even minutes, creates a poor user experience that hinders real-time collaboration and prevents widespread adoption [24–26]. This performance bottleneck has been a significant barrier to the development of practical, decentralized software development tools.

To address this critical latency challenge, we look to established architectural patterns from the literature. Research has shown that using an off-chain "fast path" or cache is a recognized strategy for enhancing the performance of blockchain systems [27–29]. Similarly, the use of cryptographic techniques like Shamir's Secret Sharing (SSS) has been validated in other high-security domains to distribute trust and eliminate single points of failure [30–34]. Building upon these validated concepts, we propose a novel hybrid architecture that makes two core contributions:

1. We introduce a trust-minimized security model specifically for VCS that uses SSS to distribute the cryptographic repository key.
2. We design an authoritative-first, optimistic-fallback protocol that uses a lightweight middleware to make the user workflow highly responsive, effectively hiding the blockchain's transaction delay from the end-user.

We have implemented this system as a full-stack decentralized application and conducted a comprehensive performance evaluation on the public Sepolia testnet. Our results demonstrate that this architecture not only provides a secure and auditable platform but also achieves highly competitive performance. Most notably, we show that our system's user-perceived push time can be faster than a standard 'git push' for common repository sizes, proving that a carefully designed decentralized VCS can balance robust security with the practical need for speed and efficiency. This paper is organized as follows. The Related works section reviews the literature on decentralized version control and relevant architectural patterns. The Materials and methods section details our proposed system architecture and protocols. The Results and discussion section presents our experimental results and a detailed performance analysis. Finally, the Conclusion section concludes the paper and discusses future work.

Related works

This section reviews the literature across three key domains that inform our work: (1) existing decentralized version control systems and their limitations; (2) architectural patterns for managing decentralized trust and security; and (3) strategies for mitigating the performance latency of blockchain systems.

Decentralized version control: Prior foundational challenges

Several early works have explored the application of blockchain technology to version control systems (VCS). One of the first, Capivara, proposed a decentralized package version control system using a proof-of-download consensus approach [35]. While conceptually innovative, the work did not include an implementation or empirical evaluation, leaving its practical

effectiveness unexplored. Another foundational study by Nizamuddin et al. used Ethereum smart contracts and the InterPlanetary File System (IPFS) for document version control [36]. The authors demonstrated the efficiency of using IPFS for decentralized storage; however, their reliance on synchronous Ethereum transactions for every version control action highlighted the significant time delays that could be prohibitive in real-world, collaborative scenarios.

Subsequent research has built upon these foundations. Systems like BDA-SCV by Hammad et al. [37] and PineSU by Grilli and Speziali [38] have created functional systems that directly combine Git workflows with a blockchain backend to ensure data integrity and authenticity. Other works have focused on related use cases, such as providing auditable versioning for scientific and educational artifacts [39–41]. While these systems advance the field, a recurring theme is the performance trade-off, where the added security of on-chain operations often results in increased latency, underscoring the critical need for a solution that prioritizes user-perceived speed [42,43].

Other works have focused on using blockchain for intellectual property (IP) management. The application of smart contracts to protect digital music [44,45] and literary IP [46–49] showcases the potential for blockchain to provide a transparent and tamper-resistant approach to managing ownership records. However, these systems, like RecordsKeeper [50] and the solution by Eleks Labs [51], typically focus on authenticity and rights management rather than the specific, high-frequency, collaborative workflows required by a VCS. A systematic mapping study by Demi et al. on blockchain in software engineering confirmed the technology's potential but also emphasized the significant challenges of scalability and complexity that must be overcome for practical adoption [52].

Decentralized trust and key management via secret sharing

A fundamental challenge in decentralized systems is managing authority and access to shared resources without a central administrator. The literature has increasingly converged on secret sharing schemes as a powerful cryptographic primitive for distributing trust. These schemes allow a secret key, such as a master encryption key, to be split into multiple shares, requiring a threshold of participants to collaborate to reconstruct it. This approach provides a robust defense against single points of failure and malicious attacks, a principle explored in contexts ranging from generic cloud databases [34,53] to the protection of high-value crypto assets [54].

This architectural pattern has been explicitly validated in various high-stakes domains that require both high security and data integrity. In the context of the Internet of Things (IoT) and vehicular ad-hoc networks (VANETs), where ensuring trust is pivotal, combining blockchain with a secret sharing mechanism has become a state-of-the-art solution. Works by Kim et al. [32] and Mao [55] demonstrate the use of Shamir's Secret Sharing (SSS) and blockchain to secure document management and sensitive medical data on IPFS . This principle is further extended by Bansal et al. [56] and Nakkar et al. [57], who leverage SSS to build lightweight and reliable authentication schemes for resource-constrained devices like Unmanned Aerial Vehicles (UAVs).

Furthermore, a significant body of work focuses on building comprehensive, decentralized trust management frameworks. The work by Razzaq et al. is notable in this area, establishing clear precedents for using blockchain as an immutable trust anchor to coordinate interactions and manage data securely in diverse applications such as educational platforms, healthcare, and vehicular networks [58–65]. Similarly, studies on VANETs by Gazdar et al. [66],

Chen et al. [67], and Pu et al. [68] propose blockchain-based systems to verify the credibility of messages and manage trust between vehicles. These works, along with comprehensive surveys on blockchain for cybersecurity [69], confirm that combining a blockchain ledger for auditability with cryptographic techniques for distributed trust is a state-of-the-art methodology.

Architectural patterns for mitigating blockchain latency

While cryptographic mechanisms can solve for decentralized trust, they do not inherently address the performance limitations of the underlying blockchain ledger. The literature has extensively explored this challenge, converging on a primary strategy: moving the bulk of storage and computation off-chain, while using the blockchain itself as a lightweight anchor for verification and asynchronous settlement. This "off-chain first" philosophy is critical for making decentralized applications practical and responsive.

Amongst several works validating this approach, systems like Saguaro by Amiri et al. and FLCoin by Ren et al. use hierarchical or layered blockchain architectures to reduce communication overhead and consensus latency in edge computing environments [70, 71]. Others focus on optimizing the consensus layer itself, with protocols like Banyan and Remora introducing "fast paths" or "optimistic paths" to reduce block finalization time [72,73]. A particularly relevant strategy is the use of off-chain caching. Kim and Park, for instance, propose a distributed caching architecture to guarantee real-time responsiveness for DApps [74,75], while Liang et al. introduce inter-shard caching in their Sparrow protocol to expedite smart contract execution [76]. This same principle of improving performance by separating concerns is also seen in the domain of Network Function Virtualization (NFV), where service chains are orchestrated to balance latency and resource costs [24,77–79].

Our system applies this validated "off-chain fast path" pattern directly to the version control workflow. The middleman component in our architecture functions as a specialized, temporary cache for key shares, analogous to the off-chain layers in the aforementioned systems. This design choice allows us to optimize for user-perceived latency, ensuring that developer workflows are not blocked by on-chain confirmation times. Our asynchronous design achieves a highly responsive user experience, a significant advancement for usable decentralized applications.

Materials and methods

In this section, we describe our proposed method, which uses the Ethereum blockchain, the InterPlanetary File System (IPFS), and a hybrid cryptographic approach to authorize, monitor, and maintain version control for code repositories. A key innovation of our system is the elimination of a single point of trust in the key management process through the use of Shamir's Secret Sharing (SSS), which allows us to distribute trust between the blockchain and a temporary centralized middleware. This architecture allows for the secure sharing and tracking of different versions of code while mitigating the inherent latency of public blockchains.

Push process

The repository **push process**, as illustrated in Fig 1, is designed to be asynchronous to optimize the user experience. It begins with the client-side encryption of the user's source code. The encrypted repository is uploaded to a decentralized storage provider, generating a unique

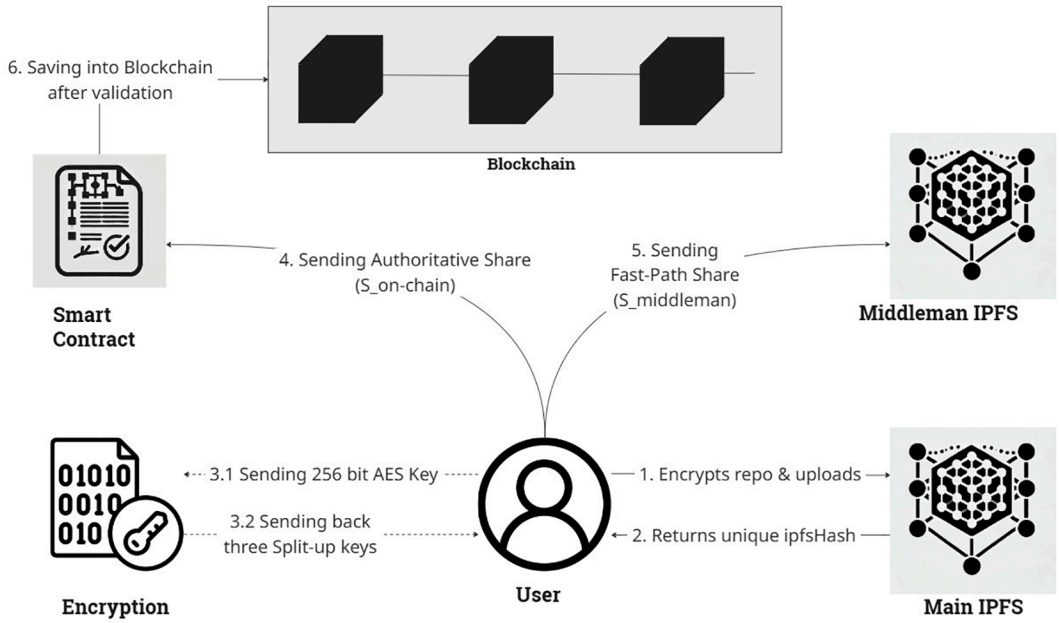

Fig 1. Proposed system push process: Source code is encrypted and stored on IPFS. The encryption key is split into three shares via SSS: One is retained by the owner, one is sent to the middleman for fast retrieval, and one is registered on the blockchain asynchronously.

https://doi.org/10.1371/journal.pone.0331131.g001

IPFS Content Identifier (CID). To secure the repository's encryption key, we employ a (k,n)-threshold secret sharing scheme. The resulting key shares are strategically distributed: one share is sent to a temporary middleman for optimistic fallback retrieval; another is registered on the Ethereum blockchain as the authoritative, long-term record in a background transaction; and the final share is retained by the repository owner.

- **Client-Side Encryption and IPFS Upload**: The process initiates on the user's client machine. The source code repository (e.g., a .zip file) is encrypted using a newly generated 256-bit AES symmetric key. This encrypted data blob is then uploaded to a decentralized storage service compatible with IPFS, such as Pinata, which returns a unique and immutable IPFS CID. This CID serves as the permanent address for the encrypted repository.
- **Key-Share Generation via Shamir's Secret Sharing (SSS)**: To address the trust and latency issues of storing a complete key, we enhance our security model with SSS. The 256-bit AES key (bundled with its Initialization Vector) is treated as a single secret. This secret key is then split into n unique shares using a (k,n)-threshold scheme (e.g., $k = 2, n = 3$). In this scheme, any k shares can reconstruct the original secret, but any $k–1$ shares reveal no information, cryptographically eliminating a single point of failure.
- **Decentralized Share Distribution**: The generated key shares are distributed to distinct entities to ensure resilience and facilitate our optimistic retrieval protocol.
  - **Owner's Share:** One share is immediately returned to the repository owner. This share acts as the primary "key" that the owner can give to collaborators to grant access.

– **Middleman's Share:** A second share is sent to a lightweight, temporary middleman middleware. This share is cached and made available for immediate retrieval, serving as the fast fallback path of our protocol.
– **On-Chain Share:** The third share is included in a transaction sent to our smart contract on the Ethereum blockchain. This serves as the authoritative, primary path—the immutable, long-term record that is queried first during retrieval.

Pull process

The repository **pull process**, as shown in Fig 2, employs an authoritative-first, optimistic-fallback model. The first step is always a non-blocking call to the smart contract to verify the collaborator's permission. Once access is granted, the system prioritizes reliability by first attempting to fetch the required key share from the authoritative on-chain source. Only if this primary path is unavailable (e.g., because the submission transaction is still pending confirmation) does the system automatically fall back to the fast middleman path. This design ensures that the most trustworthy data source is always preferred, while the middleman provides a crucial mechanism to bypass latency and ensure a responsive user experience.

- **Permission Verification**: Before any data retrieval, the application makes a view call to the smart contract's '`checkAccess`' function. This provides an immediate, authoritative check of the user's permissions. The process only continues if access is granted.
- **Authoritative Key-Share Retrieval**: The client application first calls the '`getOnChainShare`' function of the smart contract. If the transaction has been confirmed and the share is present, it is returned, and the system proceeds directly to key reconstruction.

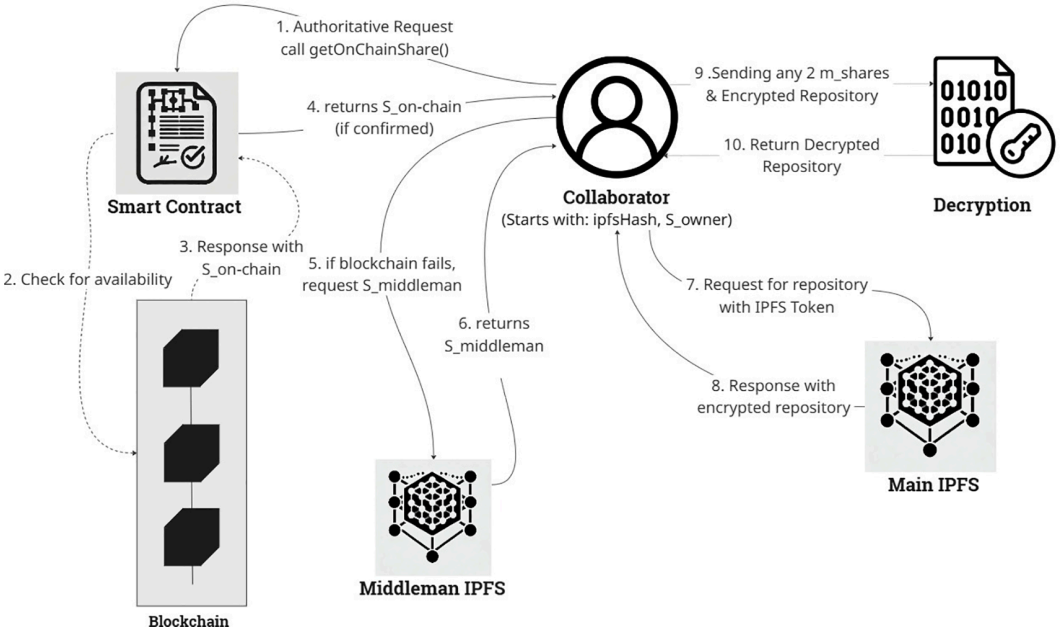

Fig 2. Proposed system pull process: After an on-chain permission check, the system first queries the authoritative blockchain for a key share. If unavailable due to latency, it optimistically falls back to the middleman, ensuring a responsive user experience.

https://doi.org/10.1371/journal.pone.0331131.g002

- **Optimistic Fallback Retrieval**: If the on-chain call fails or returns an empty value (indicating a pending transaction), the application automatically falls back to the "optimistic path." It sends a request to the middleman middleware to fetch the second required share.
- **Key Reconstruction and Decryption**: Once any two shares are successfully retrieved from either source, the client-side SSS algorithm combines them to perfectly reconstruct the original AES key and IV. The application then uses the IPFS CID to fetch the encrypted data blob from a public IPFS gateway (e.g., 'gateway.pinata.cloud'). The downloaded content is decrypted in-memory using the reconstructed key, yielding the original source code.

Environment setup and implementation

The system was developed as a client-side web application using standard web technologies (HTML, CSS, and JavaScript), leveraging the `Ethers.js` (https://docs.ethers.org/v5/) library for blockchain interaction. For decentralized storage, we integrated with the Pinata (https://www.pinata.cloud/) pinning service for uploads and a public IPFS gateway for downloads. To implement our optimistic retrieval protocol, a lightweight middleman server was developed using Node.js (https://nodejs.org/) and Express (https://expressjs.com/), and deployed as a serverless function. For testing purposes, we first used a local blockchain environment (Ganache; https://trufflesuite.com/ganache/) to validate functionality. Subsequently, the system was deployed to the public Sepolia Testnet (https://sepolia.etherscan.io/) to conduct a comprehensive performance evaluation in a real-world setting.

The development environment included the Remix IDE (https://remix.ethereum.org/) for writing and deploying our Solidity smart contract. Two Ethereum addresses were used for testing: one representing the repository owner and another for the collaborator. Each participant was provided with test Ether on the Sepolia network to facilitate transactions and validate the correctness of our cryptographic and access control mechanisms.

The implementation leverages client-side JavaScript to perform all cryptographic operations and interactions with the blockchain and middleware, ensuring that private keys and unencrypted data never leave the user's machine. The entire experimental setup was designed to rigorously test the performance trade-offs between security, decentralization, and the operational efficiency required for real-time collaboration.

Code availability. The source code for the client-side dApp, the Node.js middleman server, and the Solidity smart contract developed for this study is publicly available in a GitHub repository [80] (https://github.com/rafidhaque/Blockchain-and-middleman-ipfs-based-solution-for-repository-hosting).

Results and discussion

In this section, we present the empirical results from a comprehensive performance evaluation of our proposed system. The experiments were designed to quantify the user-perceived latency of core developer workflows, identify the underlying system bottlenecks, and provide a direct comparison against a centralized baseline (Git/GitHub). Our findings demonstrate that by architecturally decoupling the user's workflow from the inherent latency of public

Algorithm 1. Pseudocode for the smart contract.

```
Initialize State Variables:
    Mapping owners: string (ipfsHash) → address
    Mapping hasAccess: string (ipfsHash) → (address → bool)
    private Mapping onChainShares: string (ipfsHash) → string (keyShare)

procedure RegisterRepository(ipfsHash, onChainShare)
    Require: owners[ipfsHash] is not set
    owners[ipfsHash] ← msg.sender
    hasAccess[ipfsHash][msg.sender] ← true
    onChainShares[ipfsHash] ← onChainShare
end procedure

function GetOnChainShare(ipfsHash)
    Require: hasAccess[ipfsHash][msg.sender] is true
    Return: onChainShares[ipfsHash]
end function

function CheckAccess(ipfsHash, userAddress)
    Return: hasAccess[ipfsHash][userAddress]
end function

procedure AddCollaborator(ipfsHash, collaboratorAddress)
    Require: msg.sender is owners[ipfsHash]
    hasAccess[ipfsHash][collaboratorAddress] ← true
end procedure
```

Algorithm 2. Cryptographic and distribution process.

```
Input: Source Code Repository R
Output: Owner's Share S_owner, IPFS CID H_ipfs

procedure SubmitRepository(R)
    Step 1: Client-Side Encryption
    Generate a symmetric key K_sym and initialization vector IV
    R_enc ← Encrypt_AES(R, K_sym, IV)

    Step 2: Decentralized Storage
    H_ipfs ← UploadToIPFS(R_enc)

    Step 3: Key Sharing and Distribution
    Secret ← Concatenate(K_sym, IV)
    {S_owner, S_on-chain, S_middleman} ← Split_SSS(Secret, k = 2, n = 3)
    SendToMiddlemanServer(H_ipfs, S_middleman)                    ▷ Fast Path
    Call smart contract: RegisterRepository(H_ipfs, S_on-chain)   ▷ Authoritative Path
    Return S_owner, H_ipfs
end procedure
```

blockchains, our hybrid system not only provides the security benefits of decentralization but also achieves highly competitive, and in some cases, superior performance.

Performance evaluation

To quantitatively assess our system, we conducted a series of experiments on the Sepolia testnet. We defined "push latency" from a user-experience perspective: the time until a repository is available for retrieval by a collaborator. In our system, this occurs immediately after the file is uploaded to IPFS and its corresponding key share is stored in the middleman, without waiting for blockchain confirmation. All tests were repeated five times across various file sizes (1, 5, 10, and 20 MB).

Comparative analysis with centralized VCS. The primary goal of our evaluation was to contextualize our system's performance against the industry standard. Table 1 summarizes the average user-perceived latency for core operations, with Fig 3 visualizing the comparison.

The results are striking. For repositories up to 10MB, the user-perceived push time of our proposed system is faster than a standard 'git push'. This is achieved by architecturally treating the time-consuming blockchain transaction as an asynchronous background process, which allows the developer to continue their workflow without interruption. While the performance

Table 1. User-perceived performance: proposed system vs. centralized git (in seconds).

Size	System Push (s)	Git Push (s)	System Pull (s)	Git Pull (s)
1 MB	2.04	4.05	1.29	1.06
5 MB	4.19	6.16	2.41	1.07
10 MB	6.56	7.74	2.54	1.06
20 MB	11.47	8.14	4.08	1.17

https://doi.org/10.1371/journal.pone.0331131.t001

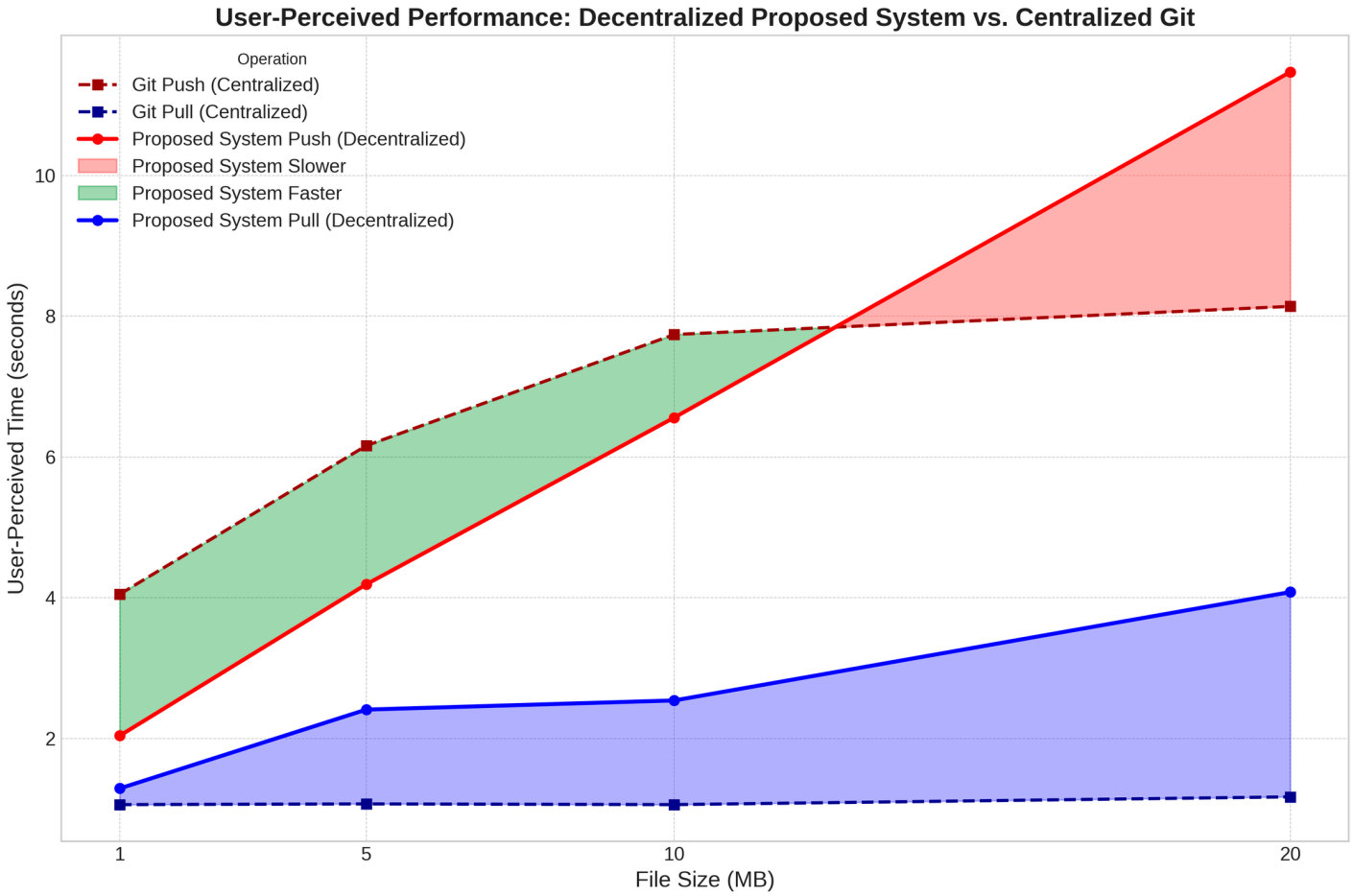

Fig 3. Performance comparison: proposed system vs. centralized git. User-perceived push time is faster than git push for common repository sizes, while pull performance is highly competitive, validating our asynchronous, optimistic design.

https://doi.org/10.1371/journal.pone.0331131.g003

for very large files (20MB) is eventually limited by the IPFS upload speed, the overall submission performance is highly competitive. Furthermore, the pull latency, leveraging the optimistic fallback to the middleman, is only marginally slower than a 'git pull', confirming the system's viability for frequent, read-heavy collaborative tasks.

System overhead and bottleneck analysis. While the user experiences a fast push, the system performs the blockchain registration in the background to ensure long-term security and auditability. Fig 4 visualizes the full system workload, including this asynchronous blockchain component.

The data clearly identifies the blockchain confirmation time as the single largest component of the total system workload, representing a "decentralization tax" of approximately 12-16 seconds per transaction. By handling this process asynchronously, our architecture provides the user with the speed of a centralized system while still gaining the security benefits of an immutable on-chain record. Furthermore, the on-chain gas cost for registering a repository was consistently measured at **206,886 gas**, confirming that our design remains cost-effective by storing only minimal data on-chain, regardless of file size.

Validation of the optimistic retrieval protocol. The effectiveness of our design hinges on our retrieval protocol, which prioritizes authority while mitigating latency. We validated

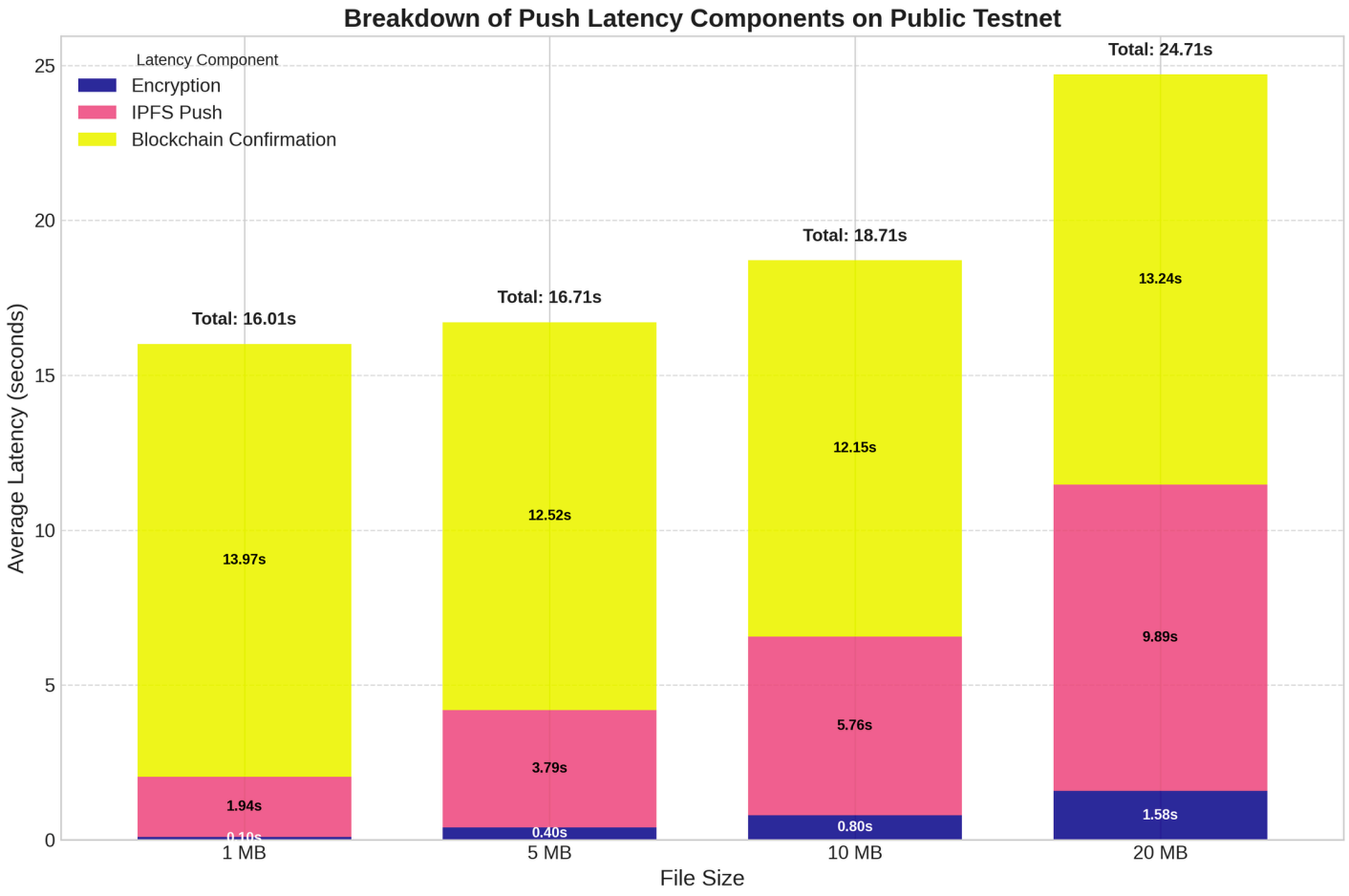

Fig 4. Breakdown of total system workload during a push operation: The `Pure Blockchain Latency` represents a large, constant overhead, confirming it as the primary system bottleneck that our asynchronous design successfully abstracts away from the user's workflow.

https://doi.org/10.1371/journal.pone.0331131.g004

this by comparing system performance in an idealized local environment against the public testnet, as shown in Fig 5.

The local testbed results establish a performance baseline for the system's core logic, free from network latency. The public testnet results confirm that our system successfully navigates real-world network delays. Crucially, our experiments demonstrated the success of the retrieval logic in both pre- and post-confirmation scenarios. When retrieval was attempted before blockchain confirmation, the authoritative on-chain call correctly failed, triggering the optimistic fallback to the middleman for the necessary key share. This allowed for immediate file access, proving the system's ability to bypass user-facing latency. Conversely, when retrieval was attempted after confirmation, the system successfully retrieved the share from the authoritative on-chain source first, never needing to contact the less-trusted middleman. This experiment empirically proves that our hybrid design effectively and intelligently decouples the user experience from blockchain finality, solving a critical usability challenge for decentralized applications.

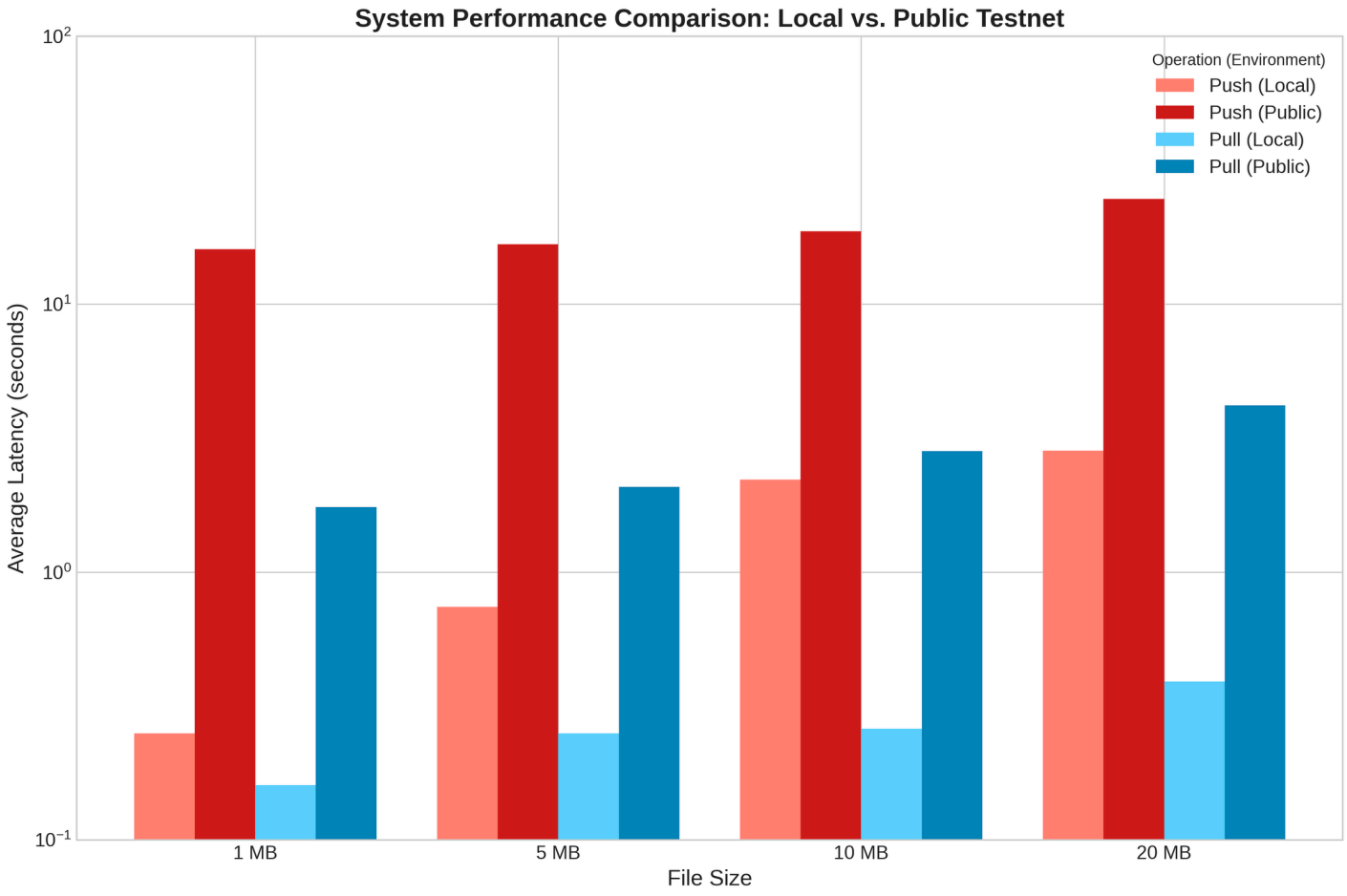

Fig 5. System latency comparison: Local vs. public testnet environments. This chart quantifies the performance overhead of public networks. Minimal local latency (lighter bars) reflects efficient core logic, while higher public latency (darker bars) is attributable to real-world network and consensus delays.

https://doi.org/10.1371/journal.pone.0331131.g005

Security and trust model

Our system's security model is designed to minimize trust and eliminate single points of failure, primarily through the integration of Shamir's Secret Sharing (SSS).

By splitting the encryption key into a (2,3)-threshold scheme, we render the centralized middleman component "cryptographically powerless." A malicious or compromised middleman only possesses one of the three shares, which, by the properties of SSS, reveals no information about the original key. An attacker would need to compromise the middleman and either the owner or the blockchain simultaneously to reconstruct the key, a significantly higher security barrier than traditional systems.

Furthermore, the authoritative copy of a key share is stored immutably on the Ethereum blockchain. This ensures that even if the temporary middleman fails or its data is lost, the repository remains permanently recoverable via the on-chain record. This design directly addresses the key retention and deletion concerns; deletion from the middleman is not a critical security requirement, as its share is both temporary for performance and insufficient for an attack. The blockchain serves as the ultimate source of truth and disaster recovery.

Developer workflow and usability

While our prototype utilizes a web-based graphical user interface, the underlying cryptographic and network operations are self-contained, making them suitable for integration into automated development pipelines. A future command-line interface (CLI) could wrap the core `push` and `pull` functions, allowing them to be scripted. This would enable the integration of our decentralized version control system into Continuous Integration/Continuous Deployment (CI/CD) workflows. For instance, a CI pipeline could be configured to automatically pull the latest version from a specific `ipfsHash`, run automated tests, and, upon success, use a securely stored owner's key share to programmatically push the new build artifacts as a new version, creating a fully decentralized and verifiable build and release process.

Conclusion

This work successfully designed, implemented, and evaluated a secure and efficient decentralized version control system that overcomes the critical latency issues plaguing many blockchain-based solutions. By integrating an authoritative-first, optimistic-fallback protocol with a Shamir's Secret Sharing security model, our hybrid architecture delivers on the promise of decentralization—providing immutable ownership records and enhanced resilience—without sacrificing the performance necessary for real-time collaboration.

Our comprehensive experiments on the public Sepolia testnet provide two key findings. First, by architecturally separating the user's interactive workflow from on-chain settlement, the user-perceived push latency of our system is highly competitive and outperforms a centralized Git/GitHub baseline for repositories up to 10MB. Second, the pull latency, which leverages the optimistic fallback mechanism, is only marginally slower than its centralized counterpart, confirming the system's viability for frequent, read-heavy developer tasks. We have empirically demonstrated that the primary system overhead is the asynchronous blockchain confirmation time, a "decentralization tax" that our architecture successfully abstracts away from the user's critical path.

Future work can build upon this validated foundation. While our system excels at core push and pull operations, further research could explore the implementation of more complex Git-native features like branching and merging within this decentralized paradigm. Additionally, optimizing the middleman component by transitioning it to a Decentralized Autonomous Organization (DAO) for coordination could further enhance the system's resilience. Such a DAO could, for example, manage a treasury of funds to pay for IPFS pinning services or automatically prune expired key shares from a network of federated middleman nodes. Governance could be managed via tokens, where stakeholders could vote on protocol upgrades or the inclusion of new, trusted middleware providers. This would further align the system with a fully trustless ethos. Ultimately, our study proves that a thoughtfully designed hybrid system can strike an effective balance between the speed of centralized services and the robust security of decentralized technologies.

Supporting information

S1 Appendix. Detailed experimental data. This appendix contains the detailed, trial-by-trial results and summary tables from the performance evaluation for the public testnet trials. (PDF)

Author contributions

Conceptualization: Md. Rafid Haque, Md. Tahmid Islam, A.B.M. Ashikur Rahman.

Formal analysis: Md. Tahmid Islam.

Investigation: Md. Rafid Haque, Sakibul Islam Munna.

Methodology: Md. Rafid Haque, Sakibul Islam Munna, Md. Tahmid Islam.

Software: Md. Rafid Haque.

Supervision: Sabbir Ahmed, Md Mehedi Hassan Onik, A.B.M. Ashikur Rahman.

Visualization: Md. Rafid Haque, Sakibul Islam Munna.

Writing – original draft: Md. Rafid Haque, Sakibul Islam Munna.

Writing – review & editing: Sabbir Ahmed, Md Mehedi Hassan Onik, A.B.M. Ashikur Rahman.

References

1. De Alwis B, Sillito J. Why are software projects moving from centralized to decentralized version control systems? In: 2009 ICSE workshop on cooperative and human aspects on software engineering; 2009. p. 36–9. https://ieeexplore.ieee.org/document/5071408
2. Ernst M. Version control concepts and best practices; 2012. https://homes.cs.washington.edu/mernst/advice/version-control.html
3. Vaidya S, Torres-Arias S, Curtmola R, Cappos J. Commit signatures for centralized version control systems. In: IFIP international information security conference; 2019. https://api.semanticscholar.org/CorpusId:189926762
4. Nakamoto S. Bitcoin: A peer-to-peer electronic cash system. Decentral Bus Rev. 2008:21260.
5. Feig E. A framework for blockchain-based applications. ArXiv; 2018. https://doi.org/abs/1803.00892
6. Wood G, et al. Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper. 2014;151:1–32.
7. Buterin V. A next-generation smart contract and decentralized application platform; 2014.

8. Kosba A, Miller A, Shi E, Wen Z, Papamanthou C. Hawk: The blockchain model of cryptography and privacy-preserving smart contracts. In: 2016 IEEE symposium on security and privacy (SP); 2016. p. 839–58. https://ieeexplore.ieee.org/document/7546538
9. Peters GW, Panayi E. Understanding modern banking ledgers through blockchain technologies: Future of transaction processing and smart contracts on the internet of money. In Banking beyond banks and money: A guide to banking services in the twenty-first century. Springer International Publishing; 2016. p. 239–78. https://doi.org/10.1007/978-3-319-42448-4_13
10. Pilkington M. Blockchain technology: principles and applications. Chapter 11. Cheltenham, UK: Edward Elgar Publishing; 2016.
11. Ruan Z. Blockchain technology for security issues and challenges in IOT. In: 2023 International conference on computer simulation and modeling, information security (CSMIS); 2023. p. 572–80. https://ieeexplore.ieee.org/abstract/document/10548025
12. Truong VT, Bao L. A blockchain-based framework for secure digital asset management. In: ICC 2023 – IEEE International conference on communications; 2023. p. 1911–6. https://ieeexplore.ieee.org/document/10279622
13. Tian F. An agri-food supply chain traceability system for China based on RFID & blockchain technology. In: 2016 13th International conference on service systems and service management (ICSSSM); 2016. p. 1–6. https://ieeexplore.ieee.org/document/7538424
14. Farooq MS, Ansari ZK, Alvi A, Rustam F, Díez IDLT, Mazón JLV, et al. Blockchain based transparent and reliable framework for wheat crop supply chain. PLoS One. 2024;19(1):e0295036. https://doi.org/10.1371/journal.pone.0295036 PMID: 38206967
15. Yang W, Xie C, Ma L. Dose blockchain-based agri-food supply chain guarantee the initial information authenticity? An evolutionary game perspective. PLoS One. 2023;18(6):e0286886. https://doi.org/10.1371/journal.pone.0286886 PMID: 37384756
16. Al-Swidi AK, Al-Hakimi MA, Al Halbusi H, Al Harbi JA, Al-Hattami HM. Does blockchain technology matter for supply chain resilience in dynamic environments? The role of supply chain integration. PLoS One. 2024;19(1):e0295452. https://doi.org/10.1371/journal.pone.0295452 PMID: 38181027
17. Zhu Y, Liu Z, Wang Z, Yang L, Peng K, Liu K. Tobacco traceability and storage scheme based on IPFS+ consortium Chain. In: 2022 2nd International conference on bioinformatics and intelligent computing; 2022. p. 235–40. https://doi.org/10.1145/3523286.3524547
18. Chenthara S, Ahmed K, Wang H, Whittaker F, Chen Z. Healthchain: A novel framework on privacy preservation of electronic health records using blockchain technology. PLoS One. 2020;15(12):e0243043. https://doi.org/10.1371/journal.pone.0243043 PMID: 33296379
19. Chenthara S, Ahmed K, Wang H, Whittaker F, Chen Z. Healthchain: A novel framework on privacy preservation of electronic health records using blockchain technology. PLoS One. 2020;15(12):e0243043. https://doi.org/10.1371/journal.pone.0243043 PMID: 33296379
20. Capece G, Lorenzi F. Blockchain and healthcare: Opportunities and prospects for the EHR. Sustainability; 2020.
21. Sun J, Ren L, Wang S, Yao X. A blockchain-based framework for electronic medical records sharing with fine-grained access control. PLoS One. 2020;15(10):e0239946. https://doi.org/10.1371/journal.pone.0239946 PMID: 33022027
22. Fan K, Wang S, Ren Y, Li H, Yang Y. MedBlock: Efficient and secure medical data sharing via blockchain. J Med Syst. 2018;42(8):136. https://doi.org/10.1007/s10916-018-0993-7 PMID: 29931655
23. Jayabalan J, Jeyanthi N. Scalable blockchain model using off-chain IPFS storage for healthcare data security and privacy. J Parallel Distrib Comput. 2022;164:152–67. https://doi.org/10.1016/j.jpdc.2022.03.009
24. Sun G, Li Y, Liao D, Chang V. Service function chain orchestration across multiple domains: A full mesh aggregation approach. IEEE Trans Netw Serv Manage. 2018;15(3):1175–91. https://doi.org/10.1109/tnsm.2018.2861717
25. Xu X, Sun G, Luo L, Cao H, Yu H, Vasilakos AV. Latency performance modeling and analysis for hyperledger fabric blockchain network. Inform Process Manag. 2021;58(1):102436. https://doi.org/10.1016/j.ipm.2020.102436
26. Javed F, Mangues-Bafalluy J. An empirical smart contracts latency analysis on Ethereum blockchain for trustworthy inter-provider agreements. ArXiv; 2025. https://doi.org/abs/2503.01397
27. Xu C, Zhang C, Xu J, Pei J. SlimChain. Proc VLDB Endow. 2021;14(11):2314–26. https://doi.org/10.14778/3476249.3476283
28. Abidin MHZ, Suchaad S, Yee OC, Ismail N, Abu MA. Scalable off-chain blockchain for vehicular network. In: 2022 1st International Conference on Information System & Information Technology (ICISIT); 2022. p. 397–402. http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9873098

29. Singh M, Aujla GS, Bali RS. Derived blockchain architecture for security-conscious data dissemination in edge-envisioned Internet of Drones ecosystem. Cluster Comput. 2022;25:2281–302. https://doi.org/10.1007/s10586-021-03496-5
30. Liu Y, Jia Z, Jiang Z, Lin X, Liu J, Wu Q, et al. BFL-SA: Blockchain-based federated learning via enhanced secure aggregation. J Syst Architec. 2024;152:103163. https://doi.org/10.1016/j.sysarc.2024.103163
31. Liu Y, Zhao Y. A blockchain-enabled framework for vehicular data sensing: Enhancing information freshness. IEEE Trans Veh Technol. 2024;73(11):17416–29. https://doi.org/10.1109/tvt.2024.3417689
32. Kim H. Digital document management system with distributed permission using secret sharing scheme. Seoul National University; 2020.
33. Yu K, Tan L, Yang C, Choo K-KR, Bashir AK, Rodrigues JJPC, et al. A blockchain-based Shamir's threshold cryptography scheme for data protection in industrial internet of things settings. IEEE Internet Things J. 2022;9(11):8154–67. https://doi.org/10.1109/jiot.2021.3125190
34. Tawakol AM. Combining Shamir's secret sharing scheme and symmetric key encryption to achieve data privacy in databases. Waterloo, Ontario, Canada: University of Waterloo; 2016.
35. da N Costa FZ, de Queiroz RJGB. Capivara: A decentralized package version control using blockchain. ArXiv. 2019. https://doi.org/abs/1907.12960
36. Nizamuddin N, Salah K, Azad MA, Arshad J, Rehman MH. Decentralized document version control using ethereum blockchain and IPFS. Comput Electr Eng. 2019;76:183–97.
37. Hammad M, Iqbal J, ul Hassan CA, Hussain S, Ullah SS, Uddin M. Blockchain-based decentralized architecture for software version control. Appl Sci. 2023.
38. Grilli L, Speziali P. Combining Git and blockchain for trusted information sharing. IEEE Access. 2024;12:88383–409.
39. Hernandez JA. DGChain: Data control version for trustworthy reproducibility with blockchain. In: 2024 6th international conference on blockchain computing and applications (BCCA); 2024. p. 648–53.
40. Almeida J, Amaral V. Towards trustworthy tracing responsibility of collaborative software engineering artefacts of student's software projects. In: 2022 IEEE 46th annual computers, software, and applications conference (COMPSAC); 2022. p. 151–60.
41. Härer F, Fill HG. Decentralized attestation and distribution of information using blockchains and multi-protocol storage. IEEE Access. 2022;10:18035–54.
42. Parvatha N. Implementing blockchain-enhanced version control systems to optimize software development life cycles. Int J Sci Res Manag. 2022.
43. Feig E. A framework for blockchain-based applications. ArXiv. 2018. https://doi.org/abs/1803.00892
44. Cai Z. Usage of deep learning and blockchain in compilation and copyright protection of digital music. IEEE Access. 2020;8:164144–54. https://doi.org/10.1109/access.2020.3021523
45. Meng Z, Morizumi T, Miyata S, Kinoshita H. Design scheme of copyright management system based on digital watermarking and blockchain. In: 2018 IEEE 42nd annual computer software and applications conference (COMPSAC); 2018. p. 359–64. https://ieeexplore.ieee.org/document/8377886
46. Savelyev A. Copyright in the blockchain era: Promises and challenges. Comput Law Secur Rev. 2018;34(3):550–61. https://doi.org/10.1016/j.clsr.2017.11.008
47. Liang W, Zhang D, Lei X, Tang M, Li KC, Zomaya AY. Circuit Copyright Blockchain: Blockchain-Based Homomorphic Encryption for IP Circuit Protection. IEEE Transactions on Emerging Topics in Computing. 2021;9(3):1410–1420. doi:10.1109/TETC.2020.2993032.
48. Xiao L, Huang W, Xie Y, Xiao W, Li K-C. A blockchain-based traceable IP copyright protection algorithm. IEEE Access. 2020;8:49532–42. https://doi.org/10.1109/access.2020.2969990
49. Jing N, Liu Q, Sugumaran V. A blockchain-based code copyright management system. Inform Process Manag. 2021;58(3):102518. https://doi.org/10.1016/j.ipm.2021.102518
50. RecordsKeeper – Decentralized Database for Decentralized Apps (DApps). https://www.recordskeeper.com/.
51. ELEKS Labs – Research and Development Blog. https://labs.eleks.com/.
52. Demi S, Colomo-Palacios R, Sánchez-Gordón M. Software engineering applications enabled by blockchain technology: A systematic mapping study. Appl Sci. 2021;11(7):2960. https://doi.org/10.3390/app11072960
53. Pandita S. Enhancing security in single and multi-cloud environments using Shamir's secret sharing algorithm. Powertech J. 2024;48(4):6320–34.

54. Skibinsky M, Dodis Y, Spies T, Ahmad W. Decentralized storage of crypto assets via hierarchical Shamir's secret sharing. Vault12. 2018. https://s3-us-west-1.amazonaws.com/vault12/Vault12%20Platform%20White%20Paper.pdf
55. Mao A. Using smart and secret sharing for enhanced authorized access to medical data in blockchain. Carleton University; 2020.
56. Bansal G, Sikdar B. Achieving secure and reliable UAV authentication: A Shamir's secret sharing based approach. IEEE Trans Netw Sci Eng. 2024;11(4):3598–610. https://doi.org/10.1109/tnse.2024.3381599
57. Nakkar M, AlTawy R, Youssef A. Lightweight group authentication scheme leveraging Shamir's secret sharing and PUFs. IEEE Trans Netw Sci Eng. 2024;11(4):3412–29. https://doi.org/10.1109/tnse.2024.3373386
58. Razzaq A. A Web3 secure platform for assessments and educational resources based on blockchain. Comput Applic Eng Educ. 2023;32(1). https://doi.org/10.1002/cae.22677
59. Razzaq A. Blockchain-based secure data transmission for internet of underwater things. Cluster Comput. 2022;25(6):4495–514. https://doi.org/10.1007/s10586-022-03701-4
60. Aljaloud A, Razzaq A. Modernizing the legacy healthcare system to decentralize platform using blockchain technology. Technologies. 2023;11(4):84. https://doi.org/10.3390/technologies11040084
61. Aljaloud A, Razzaq A. An innovative metric-based clustering approach for increased scalability and dependency elimination in monolithic legacy systems. Eng Technol Appl Sci Res. 2023;13(4):11375–113876.
62. Razzaq A, Altamimi AB, Alreshidi A, Ghayyur SAK, Khan W, Alsaffar M. IoT data sharing platform in Web 3.0 using blockchain technology. Electronics. 2023;12(5):1233. https://doi.org/10.3390/electronics12051233
63. Razzaq A, Mohsan SAH, Ghayyur SAK, Al-Kahtani N, Alkahtani HK, Mostafa SM. Blockchain in healthcare: A decentralized platform for digital health passport of COVID-19 based on vaccination and immunity certificates. Healthcare (Basel). 2022;10(12):2453. https://doi.org/10.3390/healthcare10122453 PMID: 36553977
64. Razzaq A, Zhang T, Numair M, Alreshidi A, Jing C, Aljaloud A, et al. Transforming academic assessment: The metaverse-backed Web 3 secure exam system. Comput Applic Eng Educ. 2024;32(6). https://doi.org/10.1002/cae.22797
65. Razzaq A, Numair M, Ahmed S, Akhtar M. Redefining healthcare data storage and access with decentralized technologies; 2025. p. 371–98.
66. Gazdar T, Alboqomi O, Munshi A. A decentralized blockchain-based trust management framework for vehicular ad hoc networks. Smart Cities. 2022.
67. Chen X, Ding J, Lu Z. A decentralized trust management system for intelligent transportation environments. IEEE Trans Intell Transp Syst. 2022;23:558–71.
68. Pu C. Blockchain-based trust management using multi-criteria decision-making model for VANETs; 2020. https://api.semanticscholar.org/CorpusId:228939188
69. Baskaran H, Yussof S, Rahim FA. A survey on privacy concerns in blockchain applications and current blockchain solutions to preserve data privacy. In: Anbar M, Abdullah N, Manickam S, editors. Advances in cyber security. Singapore: Springer Singapore; 2020. p. 3–17.
70. Amiri MJ, Lai Z, Patel L, Loo BT, Lo E, Zhou W. Saguaro: An edge computing-enabled hierarchical permissioned blockchain. In: 2023 IEEE 39th international conference on data engineering (ICDE); 2023. p. 259–72. https://api.semanticscholar.org/CorpusId:252222501
71. Ren S, Kim E, Lee C. A scalable blockchain-enabled federated learning architecture for edge computing. PLOS ONE. 2024;19(8):e0308991. doi:10.1371/journal.pone.0308991.
72. Vonlanthen Y, Albarello M, Sliwinski J, Wattenhofer R. Banyan: Fast rotating leader BFT. In: Proceedings of the 25th international middleware conference; 2024. p. 1–12. https://api.semanticscholar.org/CorpusId:266162509
73. Dai X, Li W, Wang G, Xiao J, Chen H, Li S, et al. Remora: A low-latency DAG-based BFT through optimistic paths. IEEE Trans Comput. 2025;74(1):57–70. https://doi.org/10.1109/tc.2024.3461309
74. Kim D, Park S. Blockchain-based caching architecture for DApp data security and delivery. Sensors (Basel). 2024;24(14):4559. https://doi.org/10.3390/s24144559 PMID: 39065957
75. Yamanaka H, Teranishi Y, Hayamizu Y, Ooka A, Matsuzono K, Li R. In: ICC 2022 – IEEE international conference on communications; 2022. p. 1076–81. http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9838289
76. Liang J, Yao P, Chen W, Hong Z, Zhang J, Cai T, et al. Sparrow: Expediting smart contract execution for blockchain sharding via inter-shard caching. IEEE Trans Parallel Distrib Syst. 2025;36(3):377–90. https://doi.org/10.1109/TPDS.2024.3516237

77. Sun G, Xu Z, Yu H, Chen X, Chang V, Vasilakos AV. Low-latency and resource-efficient service function chaining orchestration in network function virtualization. IEEE Internet Things J. 2020;7(7):5760–72. https://doi.org/10.1109/jiot.2019.2937110

78. Sun G, Zhu G, Liao D, Yu H, Du X, Guizani M. Cost-efficient service function chain orchestration for low-latency applications in NFV networks. IEEE Syst J. 2019;13(4):3877–88. https://doi.org/10.1109/jsyst.2018.2879883

79. Yang J, Yang K, Xiao Z, Jiang H, Xu S, Dustdar S. Improving commute experience for private car users via blockchain-enabled multitask learning. IEEE Internet Things J. 2023;10(24):21656–69. https://doi.org/10.1109/jiot.2023.3317639

80. Haque MR, Munna SI, Ahmed S. Blockchain-and-middleman-ipfs-based-solution-for-repository-hosting: v1.0.0; 2024. https://doi.org/10.5281/zenodo.15700465